

JavaScript OOP



OOP란?

Object-Oriented Programming

객체 지향 프로그래밍

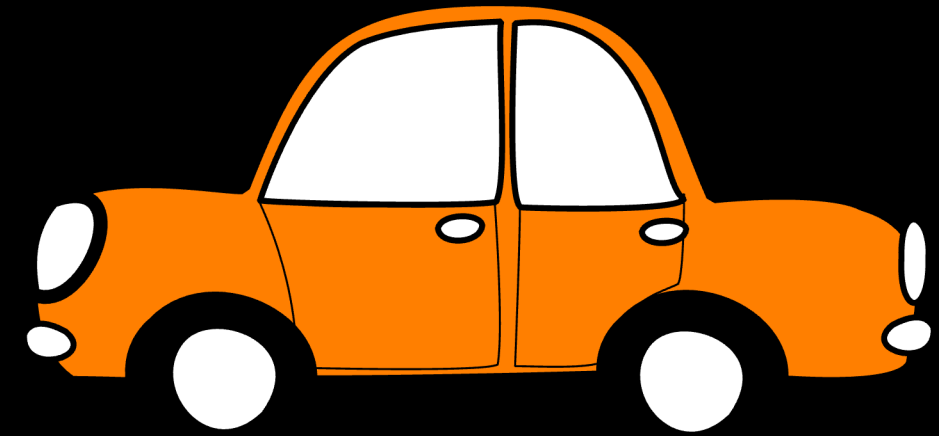
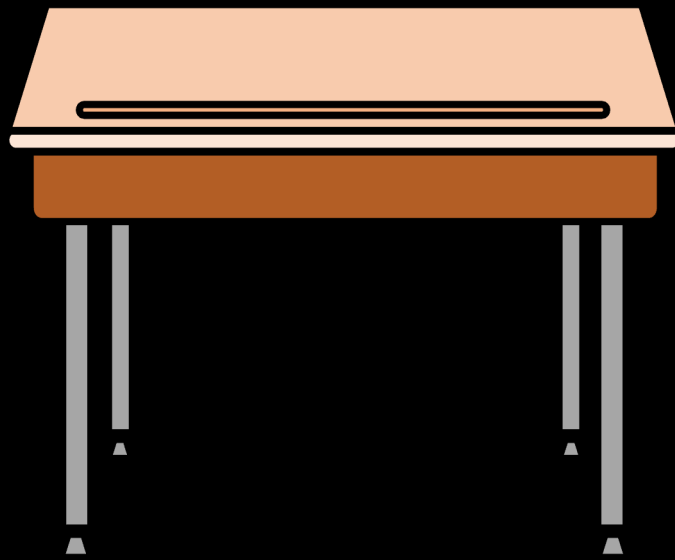


Object-Oriented Programming

프로그램을 설계할 때 **객체** 단위로 파악하고,
이들 간의 **상호작용**을 통해 기능을 구현하는
프로그래밍 패러다임



객체(Object)란?

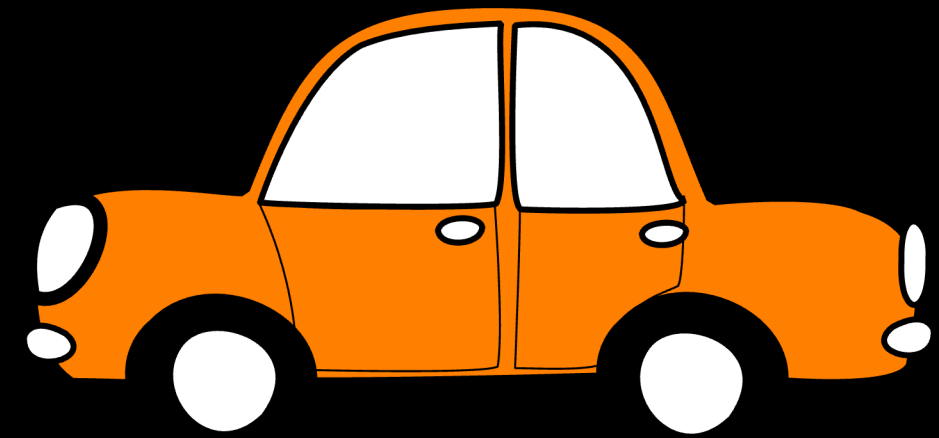
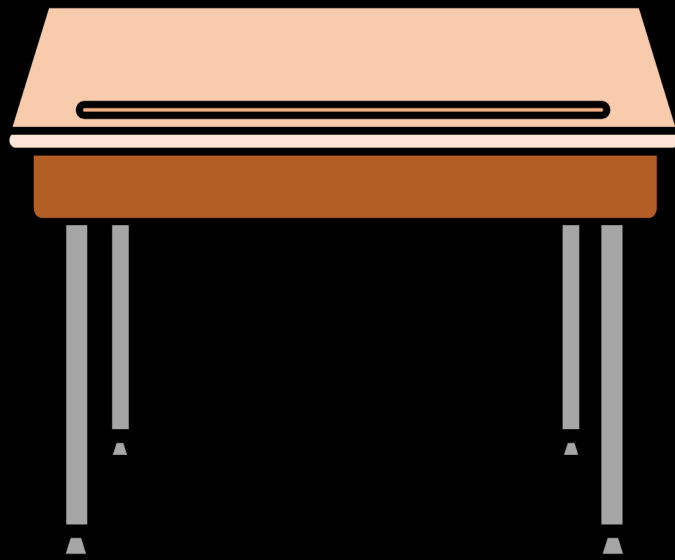


객체(Object)란 **상태**와 **동작**을 가진 사람이나 사물*이다.
일상에서 마주하는 책상, 사람, 자동차 등을 모두 객체라고 볼 수 있다.

* 사고, 문화, 철학과 같은 무형의 대상도 객체가 될 수 있다.



객체(Object)란?



즉, OOP란

우리가 일상 속에서 **사물과 사람을 바라보는 시선**으로
프로그래밍을 바라보는 것!



학습 목표



OOP를 사용하여 RPG 게임 제작을 위한 모듈*을 만들어보자!

모듈(Module): 큰 프로그램을 작은 파일들로 나눠서 관리할 수 있게 해주는 도구로, 각 파일은 특정 기능을 담당하며 필요할 때 불러와 사용할 수 있게 해준다.



OOP의 4가지 특성

추상화
Abstraction

캡슐화
Encapsulation

다형성
Polymorphism

상속성
Inheritance

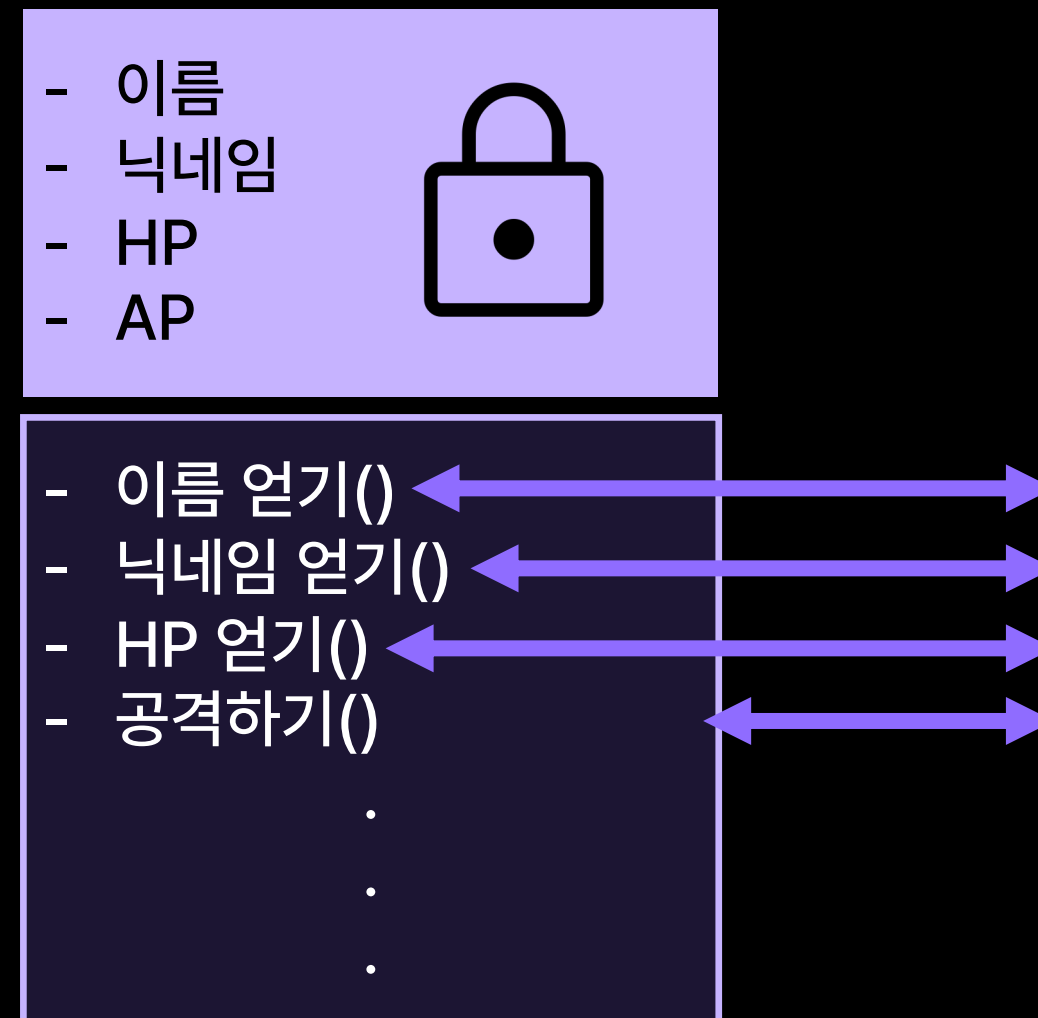
OOP의 특성: 추상화 Abstraction



많은 캐릭터들이 가진 **공통적인 특징**이 있지 않을까?

→ 그 특징들을 도출해서 묶는 것이 **추상화**!

OOP의 특성: 캡슐화 Encapsulation

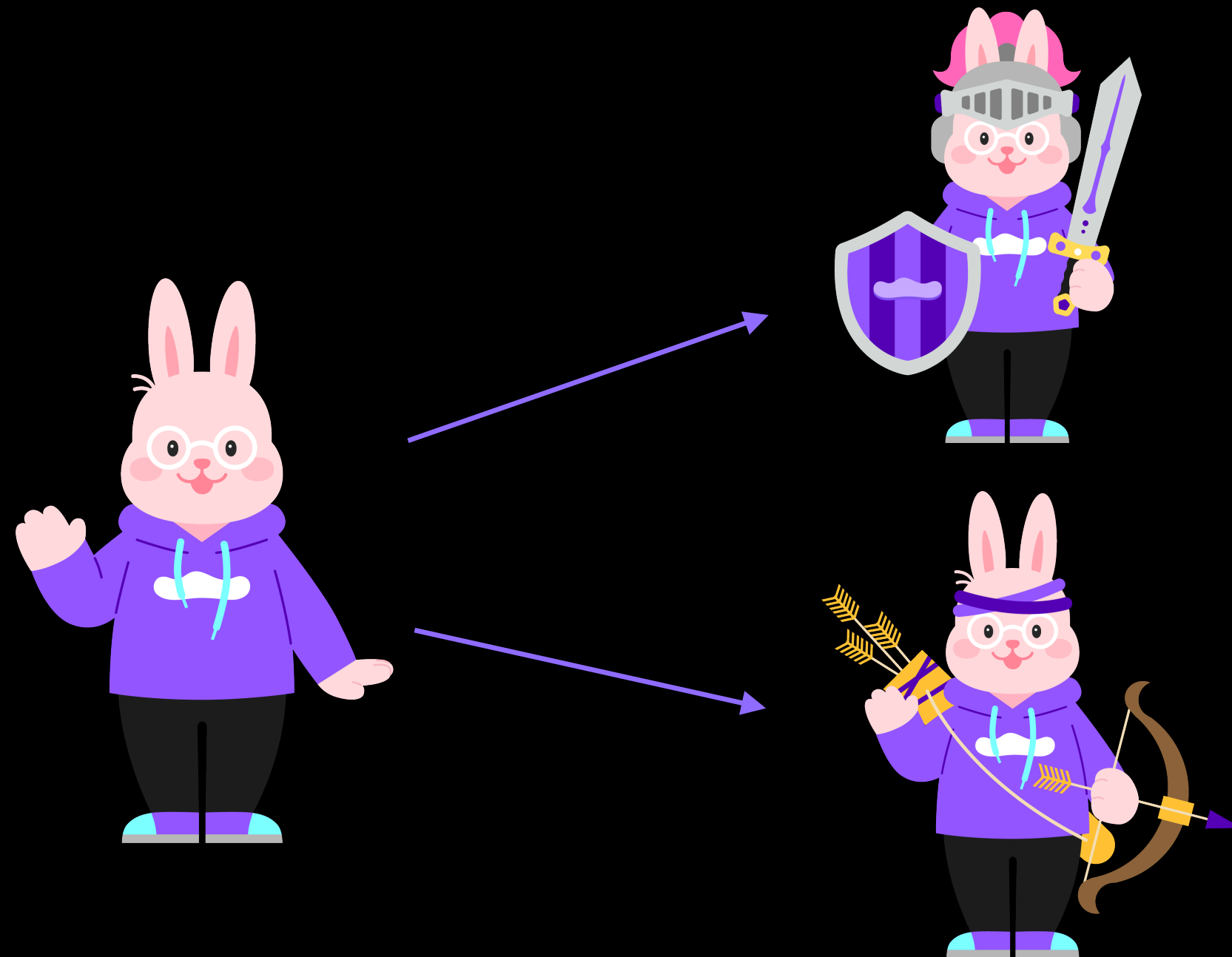


캐릭터의 정보와 동작 코드는 외부에 보이지 않는다. 외부와 상호작용 할 때에는 '메서드'를 이용한다.

→ 객체를 독립적으로 분리시키고, 외부에서 데이터와 코드에 접근하지 못하도록 하는 것이 **캡슐화**!



OOP의 특성: 상속성 Inheritance



한 캐릭터는 다양한 직업을 가질 수 있다.

이때, 각 직업은 캐릭터의 기본 특성을 **그대로 물려받고**, **추가적인 특성이나 동작**을 가진다.

→ 다른 클래스의 특징을 그대로 물려받아 새로운 클래스를 만드는 것이 **상속성**!

OOP의 특성: 다형성 Polymorphism



일반 공격

활로 쏘기



일반 공격

칼로 휘두르기



일반 공격

마법 사용하기

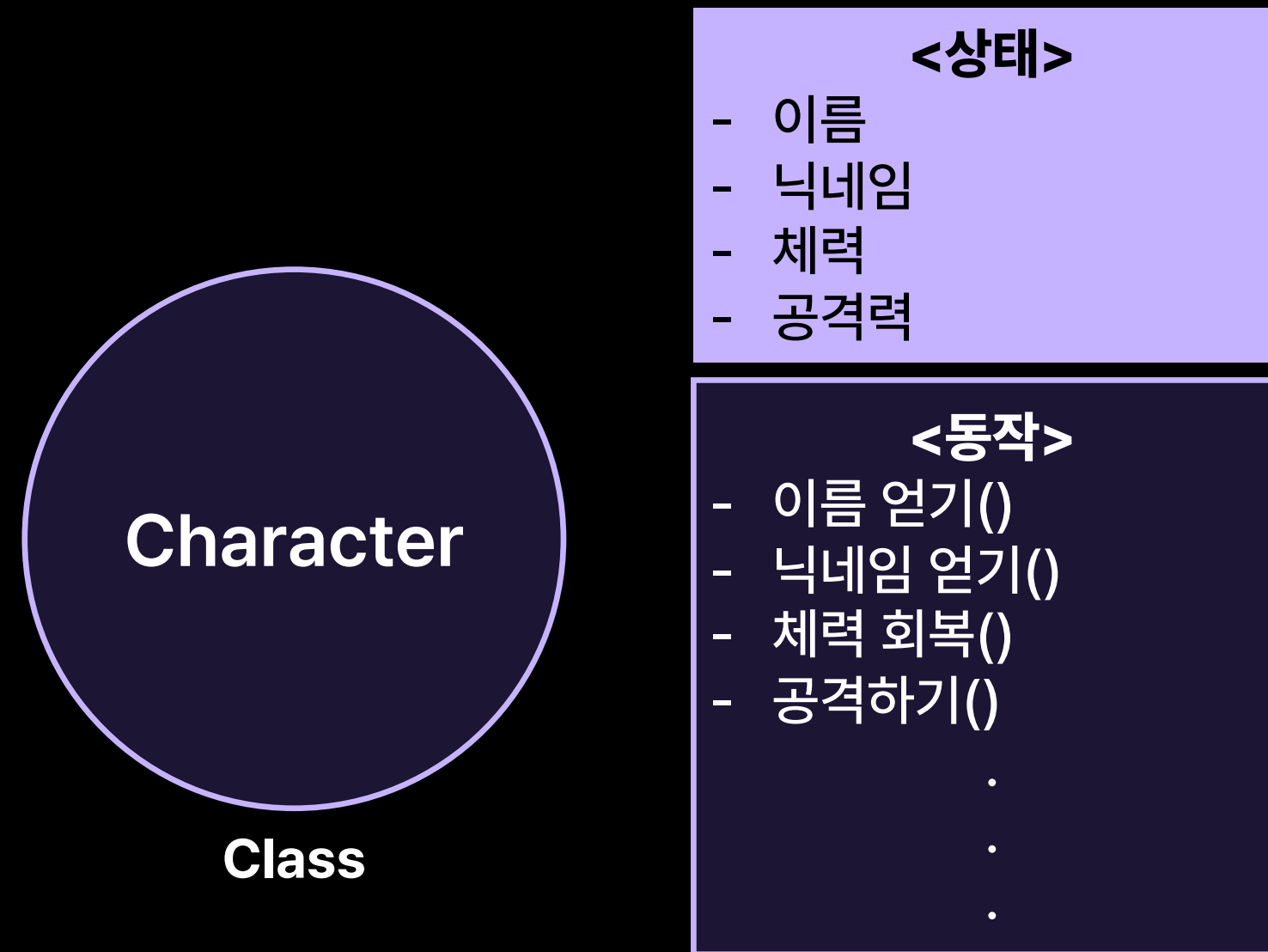
같은 이름의 다른 동작을 **여러 개 정의**하거나,

물려받은 동작을 **재정의**하여 새로운 동작을 만든다.

→ 하나의 이름을 가진 변수, 함수, 클래스가 다양한 의미로 해석될 수 있는 것이 **다형성!**



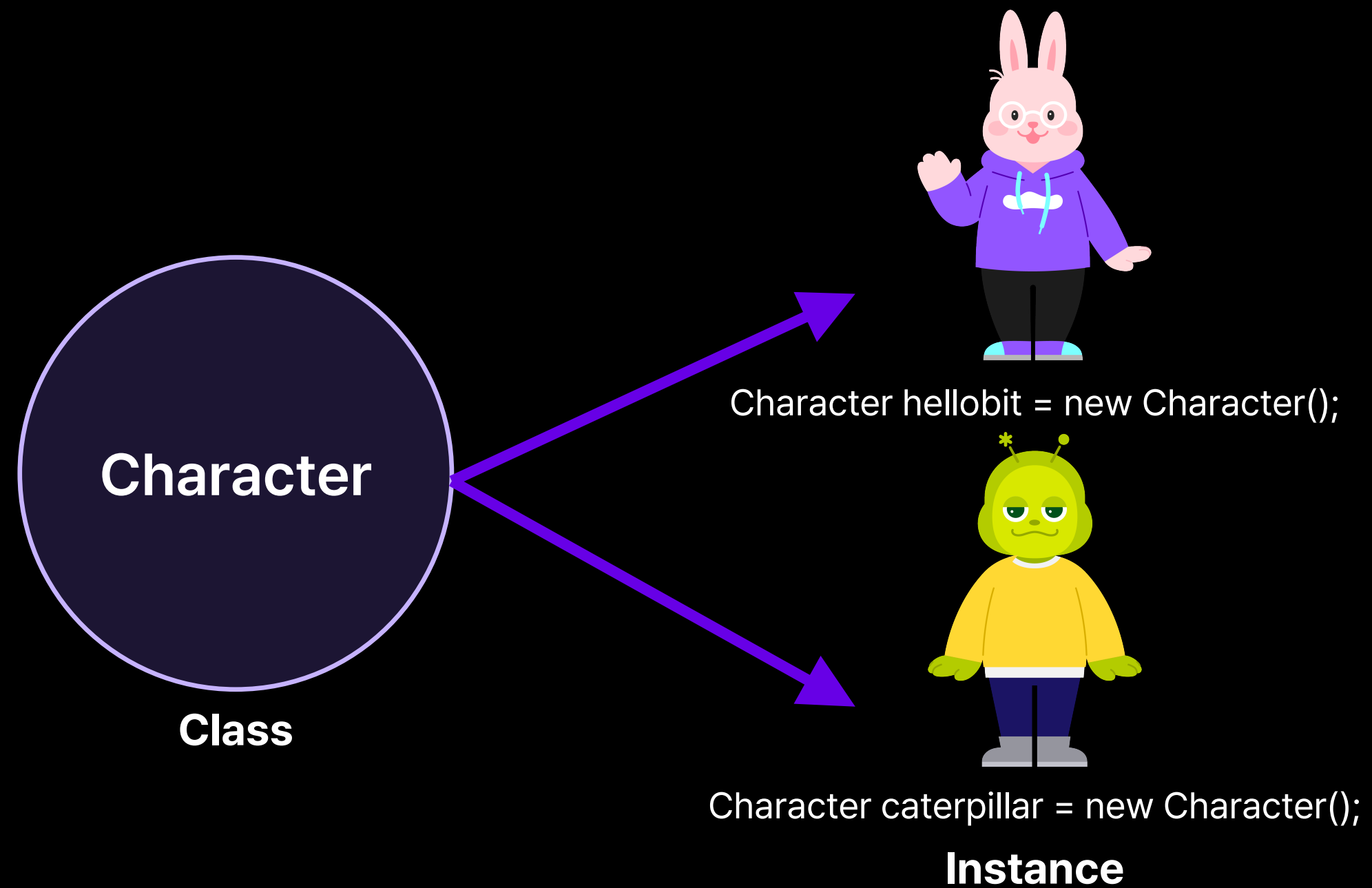
클래스 (Class)



클래스는 객체가 어떤 상태, 어떤 동작을 가져야 하는지에 대한 설계도, 설명서이다.



인스턴스 (Instance)



인스턴스는 Class에 작성된 내용을 토대로 만들어진 **실체화된 객체**이다.



클래스와 인스턴스

```
class Person {  
}  
  
const elice = new Person();
```

클래스를 선언할 때에는 **`class`** 키워드를 사용한다.

해당 클래스의 인스턴스를 생성할 때에는 **`new`** 키워드를 사용한다.



프로퍼티 (Property)



클래스는 상태와 동작을 가지고 있다.

이때, **상태**에 해당하는 것이 바로 **프로퍼티**이다.



프로퍼티 (Property)

```
class Car {  
  name;  
  speed;  
  wheels;  
}
```

프로퍼티는 *위와 같이 선언할 수 있다.

* 위 프로퍼티 선언 방식은 "클래스 필드 선언 방식"으로, ES2022부터 도입된 방법이다. 프로퍼티를 선언하는 가장 기본적인 형태는 아니다. (추후 학습)



프로퍼티 (Property)

```
class Car {  
    name;  
    speed;  
    wheels = 4;  
}
```

프로퍼티의 초기값을 설정할 수도 있다.



프로퍼티 (Property)

```
class Car {  
  name;  
  speed;  
  wheels = 4;  
}  
  
const casper = new Car();  
casper.name = "Casper";  
casper.speed = 200;  
  
console.log(casper.name); // 'Casper'  
console.log(casper.speed); // 200  
console.log(casper.wheels); // 4
```

인스턴스의 프로퍼티에 접근하려면
객체와 동일하게 프로퍼티 접근자(`.`)를 사용한다.



메서드 (Method)



게임 캐릭터는 스킬을 사용해 다른 캐릭터를 공격하거나, 아이템을 줍거나, 경험치를 올리는 등의
특정한 **동작**을 할 수 있다.



메서드 (Method)



메서드란, 객체가 행할 수 있는 이러한 동작에 대한 내용을
코드로 작성해 놓은 것이다.



메서드 (Method)

```
class Car {  
  name;  
  speed;  
  wheels = 4;  
  
  drive() {  
    console.log("driving...");  
  }  
}  
  
const casper = new Car();  
casper.drive(); // 'driving...'
```

클래스 내부에서 일반 함수를 작성하는 것과 비슷하게 선언한다.



메서드 (Method)

```
class Car {  
  name;  
  speed;  
  wheels = 4;  
  
  drive = () => {  
    console.log("driving...");  
  }  
}  
  
const casper = new Car();  
casper.drive(); // 'driving...'
```

화살표 함수를 이용해 메서드를 작성할 수도 있다.

(*this 바인딩에서 차이가 있다.)



그렇다면 우리가 제작하는 게임 캐릭터는 어떤 프로퍼티와 메서드가 들어가야 할까?

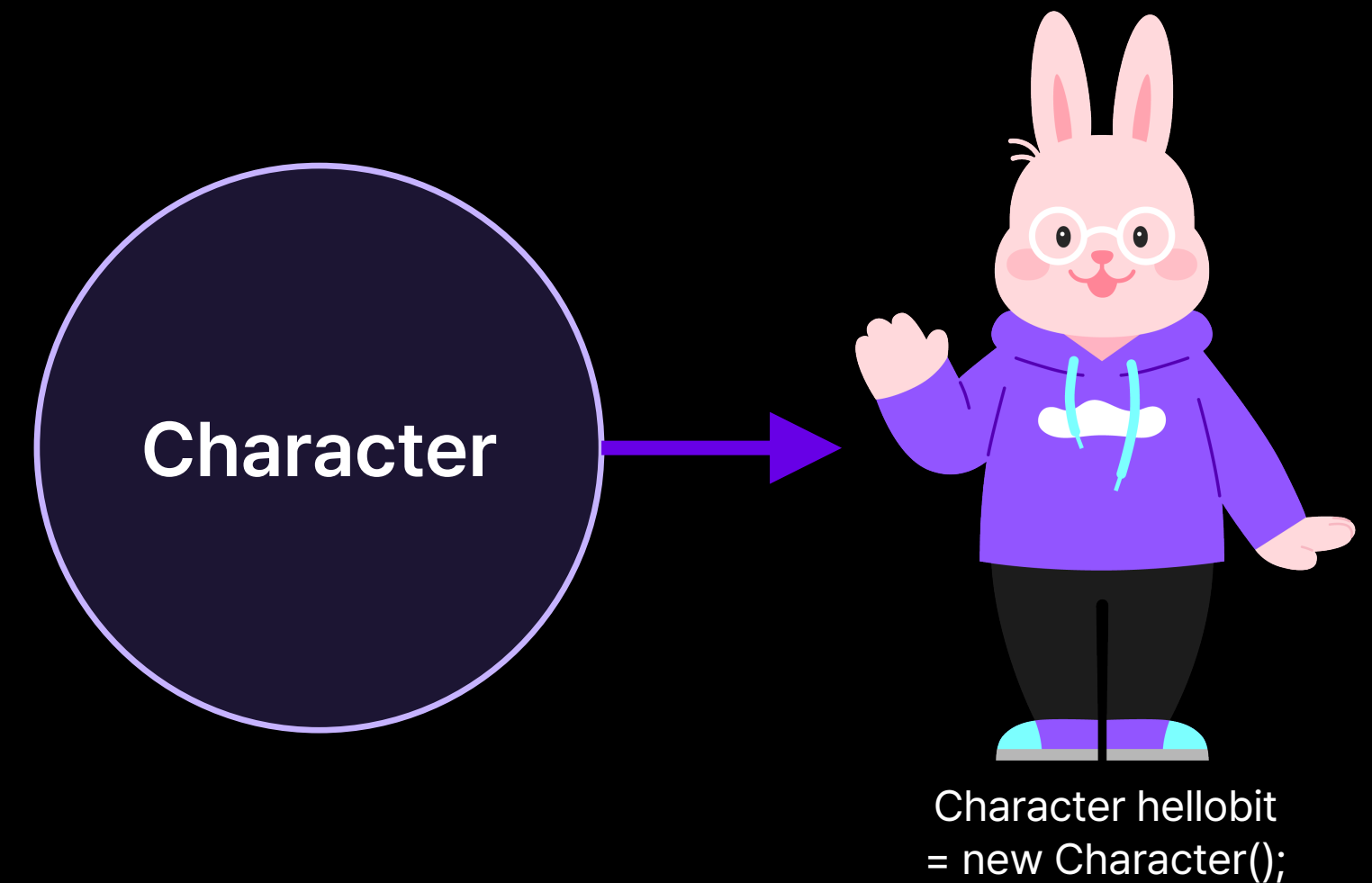
게임 캐릭터는 어떤 상태/동작을 가져야 할까?



추상화



추상화를 위해 우리는
캐릭터에 들어갈 특징에 대해 생각해 보았다.
(구체 → 추상 : 귀납적*)



그것을 클래스에 모아 추상화하였고,
인스턴스를 생성하여 추상화된 객체를 구체화시켰다.
(추상 → 구체 : 연역적*)



생성자 (Constructor)

```
class Car {  
  name;  
  speed;  
  wheels = 4;  
}  
  
const casper = new Car();  
casper.name = "Casper";  
casper.speed = 200;  
  
console.log(casper.name); // 'Casper'  
console.log(casper.speed); // 200  
console.log(casper.wheels); // 4
```

Car 인스턴스의 프로퍼티를 한 번에 지정하는 방법은 없을까?



생성자 (Constructor)

닉네임을 입력해주세요.

헬로빗

이미 존재하는 닉네임입니다.

확인

우리가 게임 캐릭터를 처음 만들 때 닉네임, 나이, 외형 등을 입력하듯이,
객체도 처음 생성될 때 **필요한 기본 정보를 세팅하는 과정**이 필요하다.



생성자 (Constructor)

닉네임을 입력해주세요.

헬로빔

이미 존재하는 닉네임입니다.

확인

생성자 메서드를 통해
객체 생성시에 필요한 초기 세팅 / 동작을 정의할 수 있다!



생성자 (Constructor)

```
class Car {  
  constructor(name, speed) {  
    this.name = name;  
    this.speed = speed;  
    this.wheels = 4;  
  }  
}  
  
const casper = new Car("Casper", 200);  
console.log(`My car is ${casper.name}.`) // 'My car is Casper.'
```

``**constructor**``라는 이름의 메서드를 선언하면
해당 메서드가 생성자가 된다.



생성자 (Constructor)

```
class Car {  
  constructor(name, speed) {  
    this.name = name;  
    this.speed = speed;  
    this.wheels = 4;  
  }  
}  
  
const casper = new Car("Casper", 200);  
console.log(`My car is ${casper.name}.`) // 'My car is Casper.'
```

`constructor` 내부에서는
프로퍼티 선언 및 초기화, 그 외의 초기화 작업 등이 이루어진다.



생성자 (Constructor)

```
class Car {  
  constructor(name, speed) {  
    this.name = name;  
    this.speed = speed;  
    this.wheels = 4;  
  }  
}  
  
const casper = new Car("Casper", 200);  
console.log(`My car is ${casper.name}.`) // 'My car is Casper.'
```

생성자는 `new` 키워드를 이용한 인스턴스 생성 시에 **자동으로 호출**된다.



```
class Car {}  
  
// 아래와 동일  
class Car {  
    constructor() { }  
}
```

‘constructor’ 메서드를 선언하지 않아도 인스턴스 생성은 가능하다.
생성자가 생략되면 **비어있는 ‘constructor’가 기본으로 생성**되기 때문이다.



생성자에서의 프로퍼티 선언

```
class Car {  
  name;  
  speed;  
  wheels;  
}
```

클래스 필드 활용

```
class Car {  
  constructor(name, speed) {  
    this.name = name;  
    this.speed = speed;  
    this.wheels = 4;  
  }  
}
```

생성자 활용

앞서 배운 클래스 필드를 이용한 프로퍼티 선언 방식은 최신 문법을 이용한 것이다.

가장 기본적인 형태의 프로퍼티 선언 방식은 **생성자에서 선언하는 것**이다.

```
class Car {  
  constructor(name, speed) {  
    this.name = name;  
    this.speed = speed;  
    this.wheels = 4;  
  }  
}
```

프로퍼티 앞에 붙는 이 `this`는 무엇일까?


```
class Car {  
  constructor(name, speed) {  
    this.name = name;  
    this.speed = speed;  
    this.wheels = 4;  
  }  
  
  printThis() {  
    const name = "localName";  
  
    console.log(this);  
    console.log(this.name);  
    console.log(name);  
  }  
}  
  
const casper = new Car("Casper", 200);  
casper.printThis();
```

실행 결과

```
Car {  
  name: 'Casper',  
  speed: 200,  
  wheels: 4,  
  __proto__: {  
    constructor: f Car(),  
    printThis: f printThis()  
  }  
}  
'Casper'  
'localName'
```

‘this’는 클래스 내부에서 **인스턴스 자기 자신을 참조**하는 변수이다.

따라서 자기 자신의 프로퍼티, 메서드에 접근 할 수 있다.


```
class Car {  
  constructor(name, speed) {  
    this.name = name;  
    this.speed = speed;  
    this.wheels = 4;  
  }  
  
  printThis() {  
    const name = "localName";  
  
    console.log(this);  
    console.log(this.name);  
    console.log(name);  
  }  
}  
  
const casper = new Car("Casper", 200);  
casper.printThis();
```

실행 결과

```
Car {  
  name: 'Casper',  
  speed: 200,  
  wheels: 4,  
  __proto__: {  
    constructor: f Car(),  
    printThis: f printThis()  
  }  
}  
'Casper'  
'localName'
```

클래스 내부에서 **프로퍼티는 항상 `this`를 통해 참조되어야 한다.**

`this`를 사용하지 않으면, 동일한 이름의 지역 변수 / 매개 변수를 참조하게 된다.

```
class Car {
  constructor(name, speed) {
    this.name = name;
    this.speed = speed;
    this.wheels = 4;
  }

  printThis() {
    ...
  }

  example() {
    printThis();
  }
}

const casper = new Car("Casper", 200);
casper.example(); // ReferenceError: printThis is not defined
```

메서드 또한 클래스 내부에서는 **항상 `this`를 통해 참조**되어야 한다.

```
class Car {  
  constructor(name, speed) {  
    this.name = name;  
    this.speed = speed;  
    this.wheels = 4;  
  }  
  
  printThis() {  
    ...  
  }  
  
  example() {  
    this.printThis();  
  }  
}  
  
const casper = new Car("Casper", 200);  
casper.example();
```

메서드 또한 클래스 내부에서는 **항상 `this`를 통해 참조**되어야 한다.



인스턴스 프로퍼티

```
class Car {  
  constructor(name, speed) {  
    this.name = name;  
    this.speed = speed;  
    this.wheels = 4;  
  }  
}
```

위와 같이 기본적인 형태로 선언한 프로퍼티는 **인스턴스 프로퍼티**라고 한다.



접근자 프로퍼티

```
class Car {
  constructor(name, speed) {
    this.name = name;
    this.speed = speed;
    this.wheels = 4;
  }

  get spec() {
    return `${this.name}/${this.speed}/${this.wheels}`;
  }

  set spec(newSpec) {
    const splitted = newSpec.split("/");
    this.name = splitted[0];
    this.speed = splitted[1];
    this.wheels = parseInt(splitted[2]);
  }
}

const casper = new Car("Casper", 200);

console.log(casper.spec); // 'Casper/200/4'
casper.spec = "Santafe/250/4";
console.log(casper.spec); // 'Santafe/250/4'
```

이와 달리, 특수한 메서드를 활용하여 프로퍼티를 선언할 수 있다.

이를 **접근자 프로퍼티**라고 한다.



접근자 프로퍼티

```
class Car {
  constructor(name, speed) {
    this.name = name;
    this.speed = speed;
    this.wheels = 4;
  }

  get spec() {
    return `${this.name}/${this.speed}/${this.wheels}`;
  }

  set spec(newSpec) {
    const splitted = newSpec.split("/");
    this.name = splitted[0];
    this.speed = splitted[1];
    this.wheels = parseInt(splitted[2]);
  }
}

const casper = new Car("Casper", 200);

console.log(casper.spec); // 'Casper/200/4'
casper.spec = "Santafe/250/4";
console.log(casper.spec); // 'Santafe/250/4'
```

‘**get**’ 키워드와 ‘**set**’ 키워드를 사용하여 메서드를 선언하면,
해당 메서드의 이름은 **프로퍼티처럼 동작**한다.



접근자 프로퍼티 - get

```
class Car {
  constructor(name, speed) {
    this.name = name;
    this.speed = speed;
    this.wheels = 4;
  }

  get spec() {
    return `${this.name}/${this.speed}/${this.wheels}`;
  }

  set spec(newSpec) {
    const splitted = newSpec.split("/");
    this.name = splitted[0];
    this.speed = splitted[1];
    this.wheels = parseInt(splitted[2]);
  }
}

const casper = new Car("Casper", 200);

console.log(casper.spec); // 'Casper/200/4'
casper.spec = "Santafe/250/4";
console.log(casper.spec); // 'Santafe/250/4'
```

`get` 키워드로 선언된 메서드는
해당 접근자 프로퍼티가 **조회될 때의 동작을 정의한다.**
= getter 메서드



접근자 프로퍼티 - set

```
class Car {
  constructor(name, speed) {
    this.name = name;
    this.speed = speed;
    this.wheels = 4;
  }

  get spec() {
    return `${this.name}/${this.speed}/${this.wheels}`;
  }

  set spec(newSpec) {
    const splitted = newSpec.split("/");
    this.name = splitted[0];
    this.speed = splitted[1];
    this.wheels = parseInt(splitted[2]);
  }
}

const casper = new Car("Casper", 200);

console.log(casper.spec); // 'Casper/200/4'
casper.spec = "Santafe/250/4";
console.log(casper.spec); // 'Santafe/250/4'
```

‘set’ 키워드로 선언된 메서드는
해당 접근자 프로퍼티 값이 **변경될 때의 동작을 정의**한다.
= setter 메서드



getter와 setter

```
class Car {
  constructor(name, speed) {
    this.name = name;
    this.speed = speed;
    this.wheels = 4;
  }

  get spec() {
    return `${this.name}/${this.speed}/${this.wheels}`;
  }

  set spec(newSpec) {
    const splitted = newSpec.split("/");
    this.name = splitted[0];
    this.speed = splitted[1];
    this.wheels = parseInt(splitted[2]);
  }
}
```

```
const casper = new Car("Casper", 200);

console.log(casper.spec); // 'Casper/200/4'
casper.spec = "Santafe/250/4";
console.log(casper.spec); // 'Santafe/250/4'
```

```
class Car {
  constructor(name, speed) {
    this.name = name;
    this.speed = speed;
    this.wheels = 4;
  }
```

```
  getName() {
    return this.name;
  }

  setName(name) {
    this.name = name;
  }
}
```

```
const casper = new Car("Casper", 200);

console.log(casper.getName()); // 'Casper'
casper.setName("Santafe");
console.log(casper.getName()); // 'Santafe'
```

OOP에서 **getter**는 **프로퍼티를 반환**해주는 메서드이고,

setter는 **프로퍼티를 변경**하는 메서드이다.

(접근자 프로퍼티 방식이 아니어도 getter와 setter 구현은 가능하다. – 오른쪽 코드 참고)



getter와 setter를 사용하는 이유

```
const casper = new Car("Casper", 200);

// 조회
console.log(casper.getName()); // 'Casper'
console.log(casper.name);      // 'Casper'

// 수정
casper.setName("Santafe");
console.log(casper.getName()); // 'Santafe'
casper.name = "Sonata";
console.log(casper.getName()); // 'Sonata'
```

그러나 직접 프로퍼티를 참조하여 조회 및 변경해도 되는데,
왜 굳이 getter와 setter 메서드를 사용하는 것일까?



getter와 setter를 사용하는 이유

```
...  
  
setSpeed(speed) {  
  if (speed < 0) throw new Error("speed cannot be minus");  
  if (typeof speed !== "number") throw new Error("speed must be number");  
  this.speed = speed;  
}  
}  
  
const casper = new Car("Casper", 200);  
casper.setSpeed(-100); // Error: speed cannot be minus  
casper.setSpeed("100"); // Error: speed must be number  
casper.setSpeed(100);
```

검증 절차 1

검증 절차 2

getter와 setter 메서드를 작성하면,
메서드를 거쳐야만 프로퍼티에 접근할 수 있도록 제어하고,
이때 검증 절차를 통해 **유효한 데이터만 저장**할 수 있도록 설계가 가능하다.

 OOP의 특성: 캡슐화 Encapsulation

Web Basic03 JavaScript 심화 –보통자료1. JavaScript OOP





- 이름
- 닉네임
- HP
- AP

- 이름 얻기()
- 닉네임 얻기()
- HP 얻기()
- 공격하기()
-
-
-



캐릭터의 정보와 동작 코드는 외부에 보이지 않는다. 외부와 상호작용 할 때에는 '메서드'를 이용한다.

→ 객체를 독립적으로 분리시키고, 외부에서 데이터와 코드에 접근하지 못하도록 하는 것이 **캡슐화**!

앞선 내용에서, 객체를 독립시키고 외부에서 데이터와 코드에 접근하지 못하도록 하는 **캡슐화**에 대해 배웠다.

getter와 setter 메서드는 OOP에서 캡슐화를 구현하는 방법이다.



```
const casper = new Car("Casper", 200);  
  
// 여전히 직접 접근 가능  
casper.speed = -100; // 객체의 무결성 보장 X
```

그러나 getter와 setter를 구현하더라도, **여전히 프로퍼티 직접 접근이 가능하다.**

→ JavaScript는 기본적으로 **완전한 캡슐화 구현이 불가능하다.**



private 프로퍼티

```
class Car {  
  #name;  
  #speed;  
  #wheels;  
  
  constructor(name, speed) {  
    this.#name = name;  
    this.#speed = speed;  
    this.#wheels = 4;  
  }  
}  
  
const casper = new Car("Casper", 200);  
console.log(casper.#name); // SyntaxError: Private name #name is not defined.
```

ES2022부터 **private 프로퍼티*** 기능이 추가되었다.

프로퍼티를 private으로 설정하면 외부에서 **직접 접근이 불가능하다!**

* 이와 같은 기능을 "접근 제어자"라고 한다. 접근 제어자는 private 외에도 public, protected가 있으나, JavaScript에서는 private과 public만 지원한다.

* JavaScript의 슈퍼셋인 TypeScript에서는 모든 접근 제어자를 지원한다.



private 프로퍼티

```
class Car {  
  #name;  
  #speed;  
  #wheels;  
  
  constructor(name, speed) {  
    this.#name = name;  
    this.#speed = speed;  
    this.#wheels = 4;  
  }  
}  
  
const casper = new Car("Casper", 200);  
console.log(casper.#name); // SyntaxError: Private name #name is not defined.
```

`#` 식별자를 사용하여 private 프로퍼티를 선언한다.

private 프로퍼티는 **클래스 필드로만 선언**할 수 있다.



private 프로퍼티 + getter, setter

```
class Car {
  #name;
  #speed;
  #wheels;

  constructor(name, speed) {
    this.#name = name;
    this.#speed = speed;
    this.#wheels = 4;
  }

  get speed() {
    return this.#speed;
  }

  set speed(speed) {
    if (speed < 0) throw new Error("speed cannot be minus");
    if (typeof speed !== "number") throw new Error("speed must be number");
    this.#speed = speed;
  }
}

const casper = new Car("Casper", 200);
console.log(casper.speed);
casper.speed = 100;
casper.speed = -100;
```

```
class Car {
  #name;
  #speed;
  #wheels;

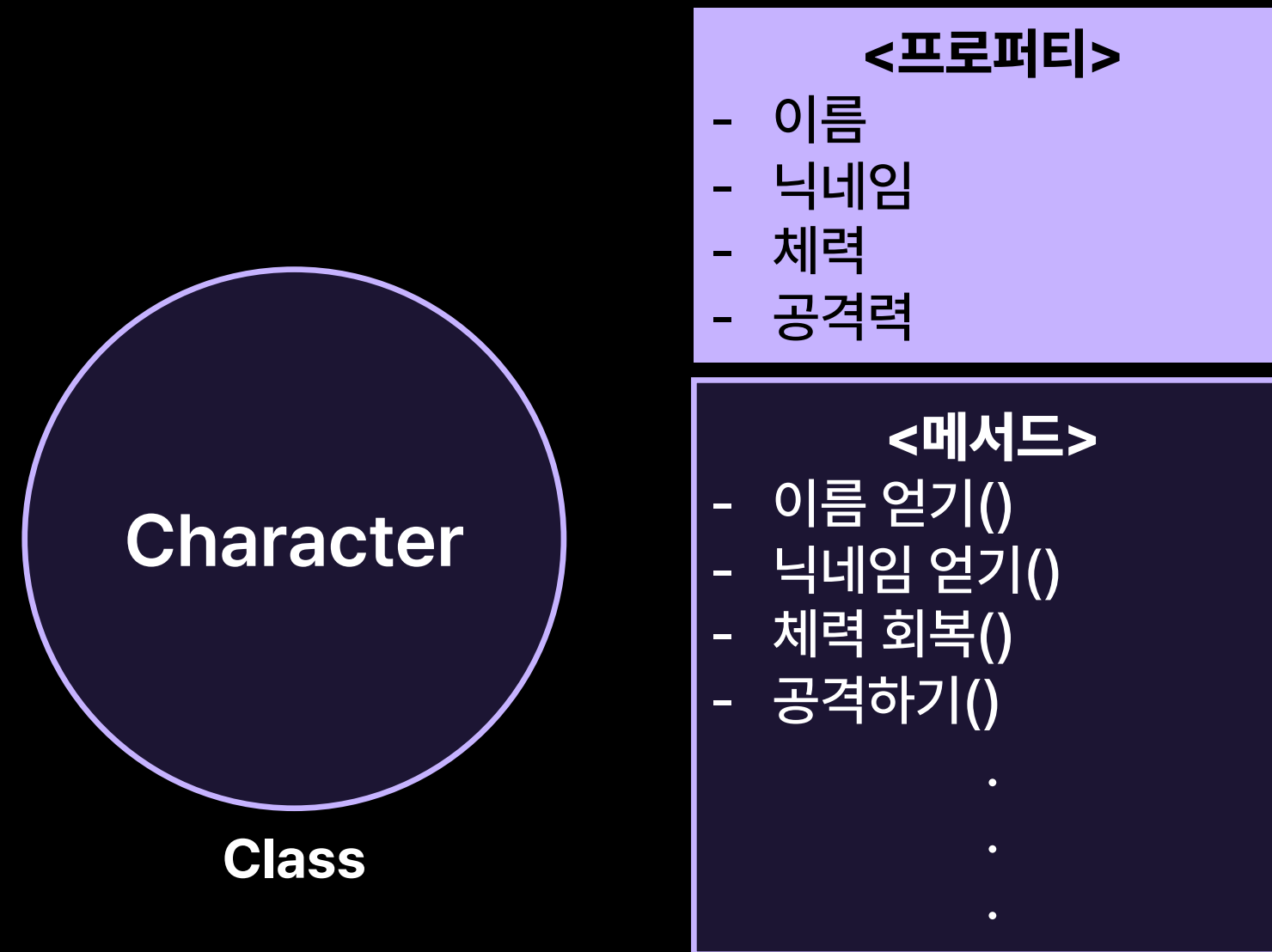
  constructor(name, speed) {
    this.#name = name;
    this.#speed = speed;
    this.#wheels = 4;
  }

  printSpec() {
    console.log(`${this.#name}/${this.#speed}/${this.#wheels}/${this.#calculateFuelEfficiency()}`);
  }

  #calculateFuelEfficiency() {
    return 100 / this.#speed;
  }
}

const casper = new Car("Casper", 200);
casper.printSpec();
casper.#calculateFuelEfficiency(); // Error
```

``#`` 식별자를 메서드에 붙이면 private 메서드를 선언할 수 있다.
private 메서드는 클래스 내부에서만 사용할 수 있다.

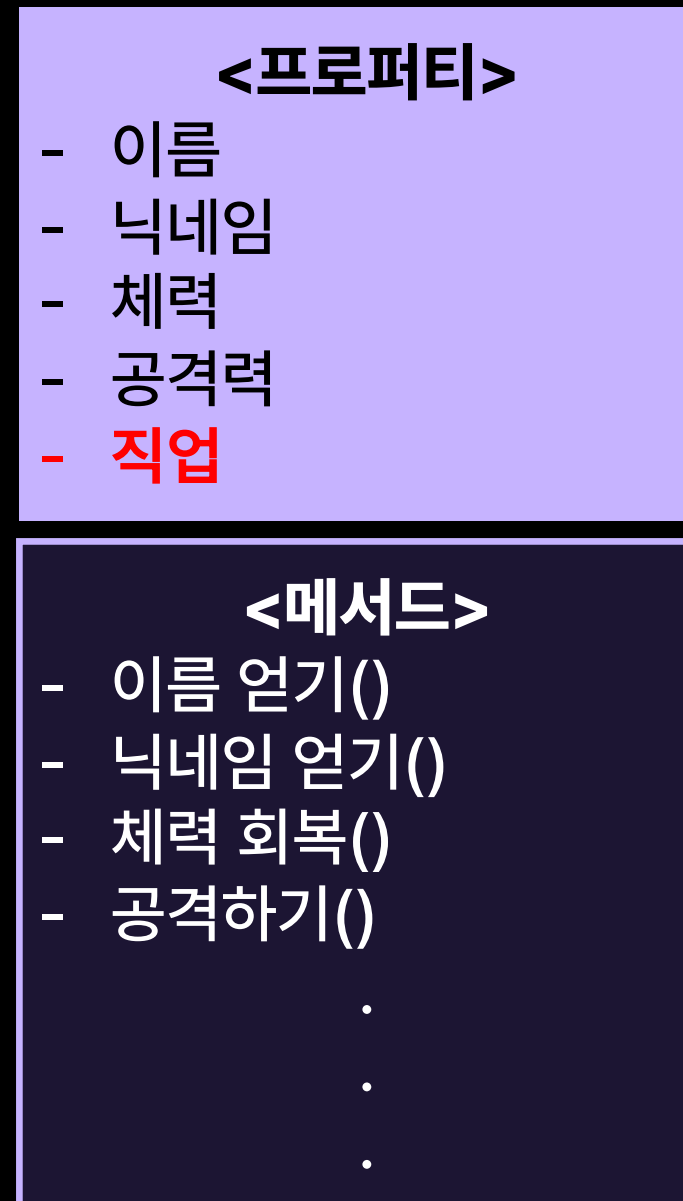


추상화를 통해 프로퍼티와 메서드를 정의하여
게임 캐릭터 클래스를 구현할 수 있다.



만약 게임 안에서 **직업**을 구현하려면 어떤 방법으로 구현해야 할까?

(직업 예시: 용사, 궁수, 마법사 등)



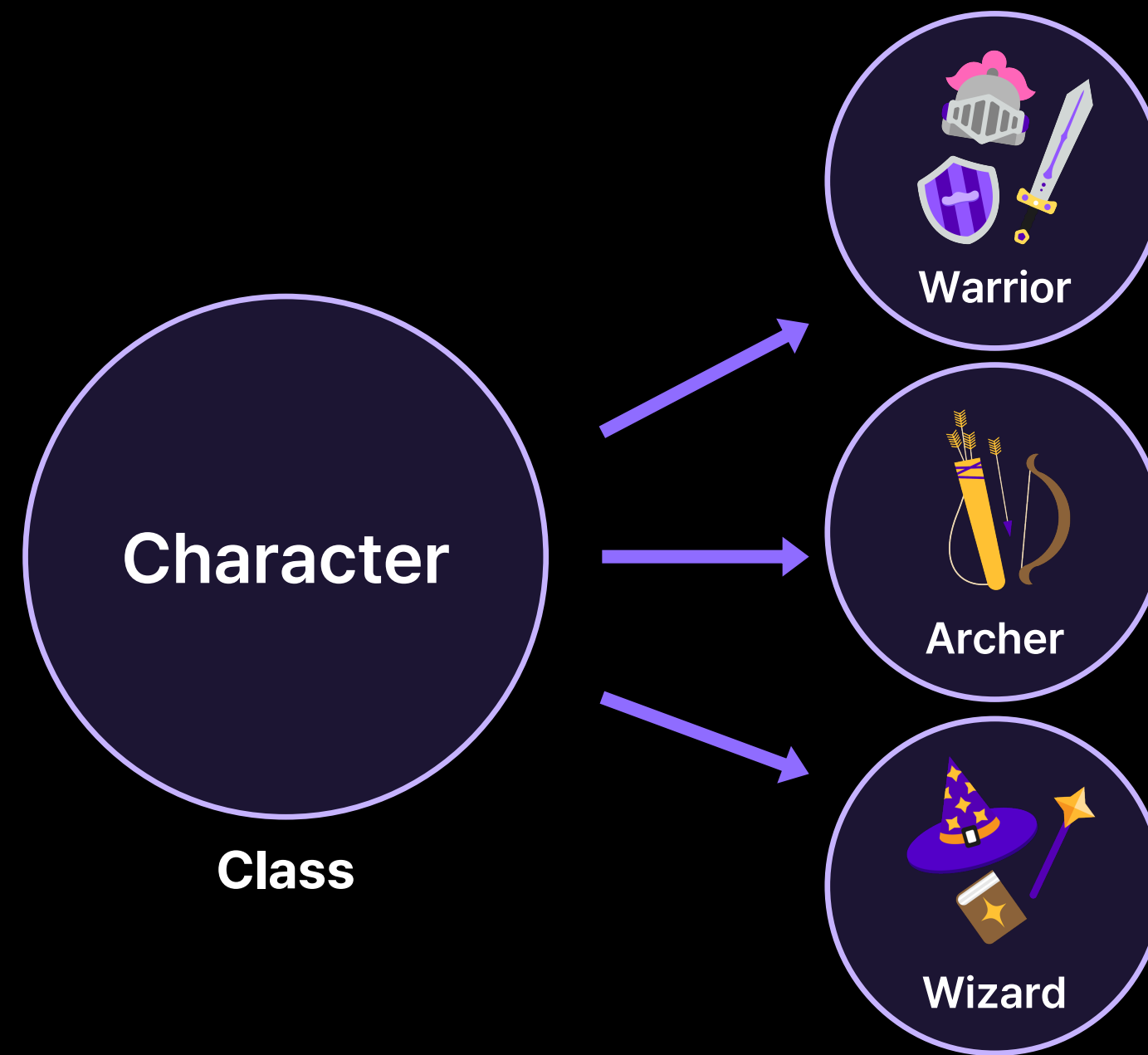
[방법 1] 프로퍼티에 직업에 대한 정보를 추가한다.

→ **문제점:** 직업 별로 추가적으로 필요한 프로퍼티, 메서드가 다를 수 있다.

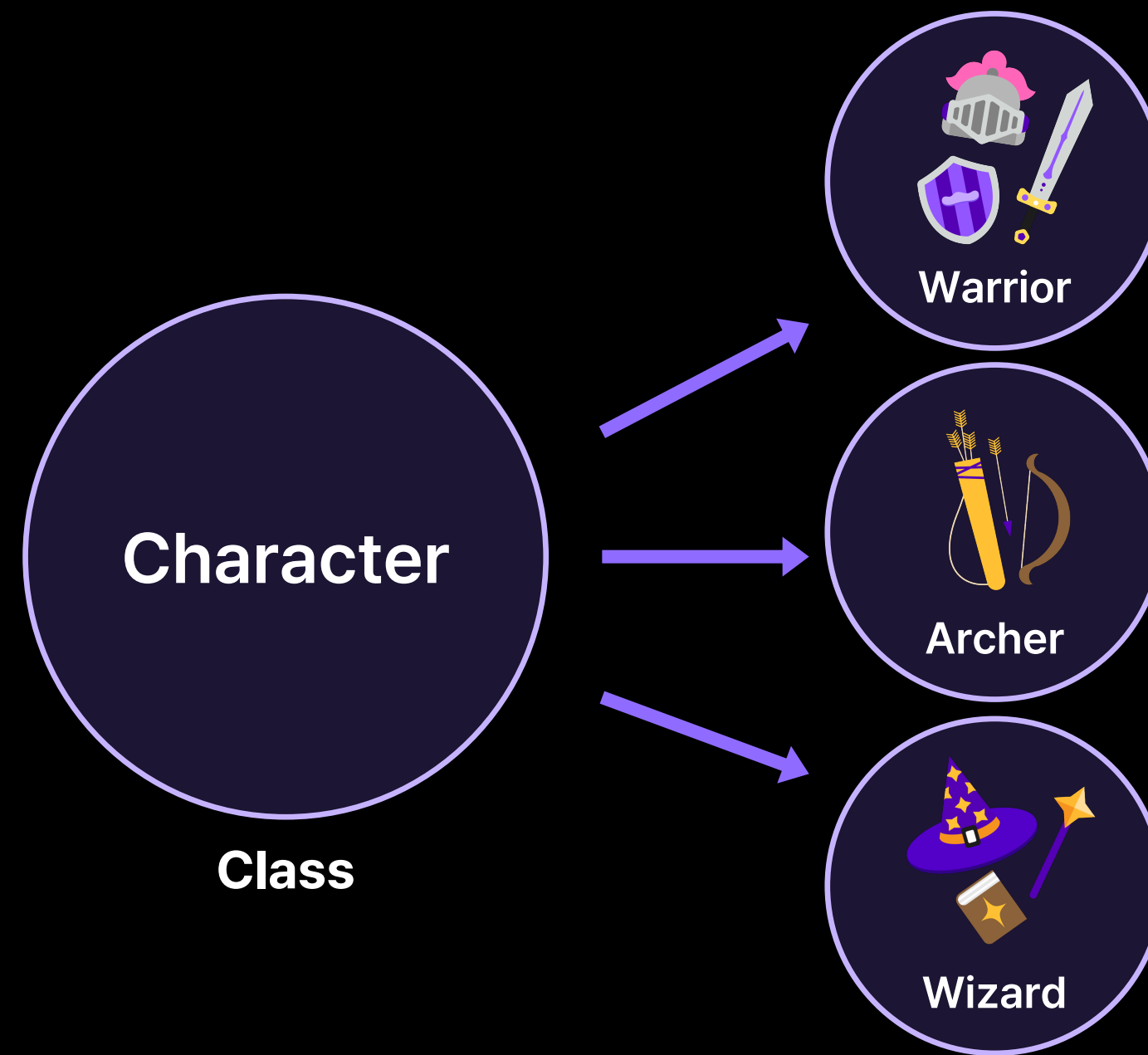


[방법 2] 캐릭터 클래스의 코드를 복사하고, 직업에 맞게 추가/수정하여 새로운 클래스를 만든다.

→ **문제점:** 코드의 중복이 발생하고, 두 클래스의 코드 상 연관성이 존재하지 않게 된다.



[방법 3] 상속을 이용하여 Character 클래스를 상속한 직업 클래스를 각각 작성한다.



상속(Inheritance)이란 기존에 존재하던 클래스의 프로퍼티와 메서드를 **그대로 물려받아**,
프로퍼티 및 메서드를 **추가**하거나 **재정의**하여 새로운 클래스를 정의하는 것이다.



상속을 활용하는 이유 1



상속을 활용하는 첫 번째 이유는,
중복되는 프로퍼티 / 메서드를 다시 작성하지 않아도 되기 때문이다.



상속을 활용하는 이유 2



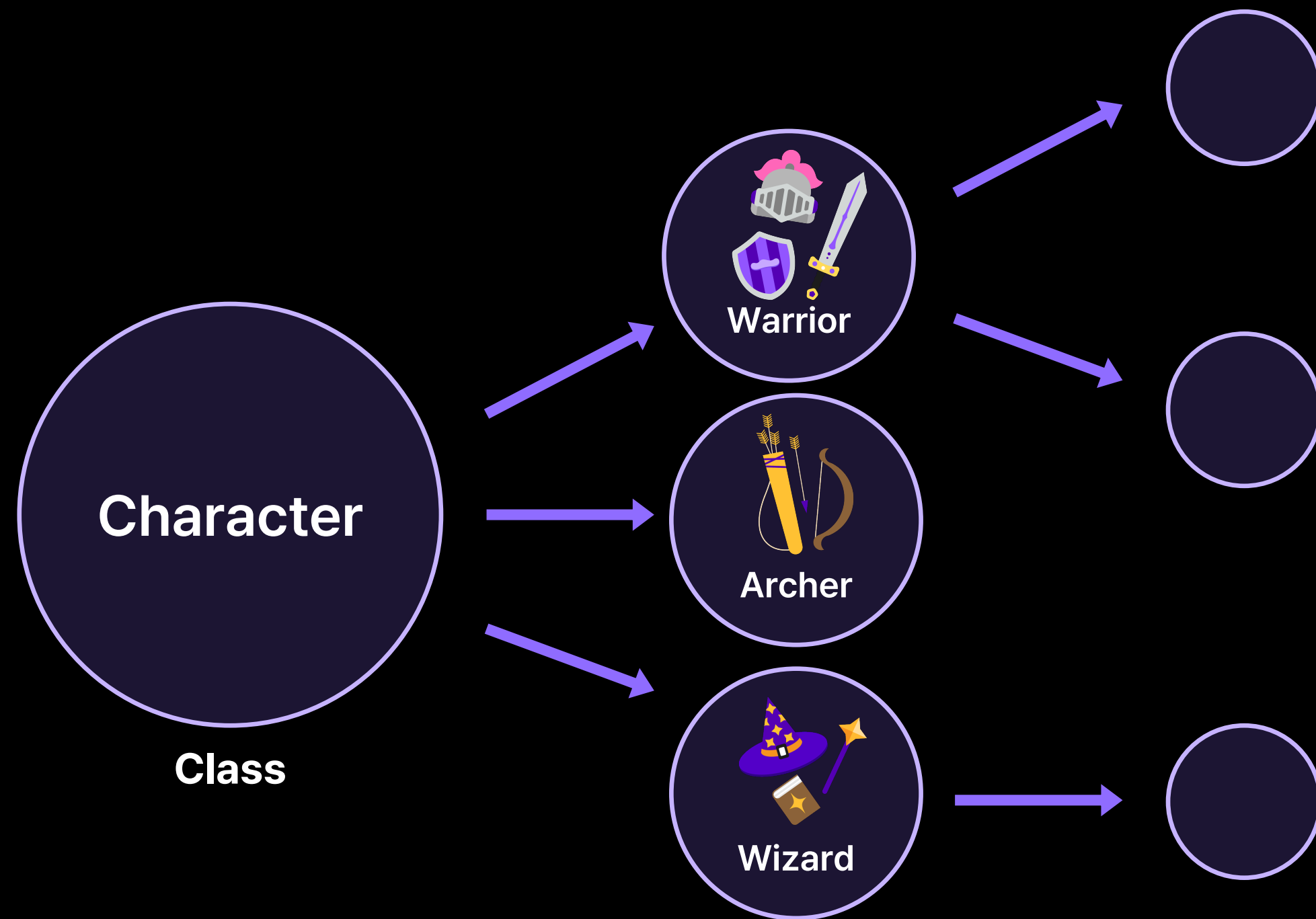
상속을 활용하는 두 번째 이유는, **클래스의 재사용성** 때문이다.

여러 클래스에 포함되는 내용을 정의해 놓으면, 각 클래스마다 상속의 형태로 재사용하여 작성할 수 있다.

이로 인해 **유지 보수가 매우 편해진다.**



상속을 활용하는 이유 3



상속을 활용하는 세 번째 이유는, **클래스 간의 계층 관계** 때문이다.

부모 클래스와 자식 클래스 간의 계층 관계를 명확히 할 수 있고, 이를 통해 **가독성**을 높이고, **다형성을 구현**할 수 있다.



상속 구현하기

```
class Car { ... }
```

부모 클래스

```
class Sedan extends Car { ... }
```

```
class SUV extends Car { ... }
```

```
class SportsCar extends Car { ... }
```

자식 클래스

‘**extends**’ 키워드를 이용해 클래스를 상속 받는다.

이때, 기존 클래스를 **부모 클래스(슈퍼 클래스)**,
상속 받는 클래스를 **자식 클래스(서브 클래스)**라고 한다.



상속 구현하기

```
class Car {  
  #name;  
  #speed;  
  #wheels;  
  
  constructor(name, speed) {  
    this.#name = name;  
    this.#speed = speed;  
    this.#wheels = 4;  
  }  
  
  printSpec() {  
    console.log(`${this.#name}/${this.#speed}/${this.#wheels}`);  
  }  
}  
  
class SportsCar extends Car { }  
  
const porsche = new SportsCar("Porsche", 400);  
porsche.printSpec(); // 'Porsche/400/4'
```

Car 클래스를 상속받은 SportsCar 클래스는 아무 내용을 작성하지 않아도,
Car 클래스의 **name, speed, age 프로퍼티, printSpec() 메서드**를 그대로 가지고 있다.



상속 구현하기

```
class SportsCar extends Car {  
  #horsePower;  
  #isTurboCharged;  
  
  boostSpeed() {  
    if (this.#isTurboCharged) {  
      console.log("Boosting speed with turbocharger!");  
      // 예시로 속도를 20% 증가시킴  
      this.#speed *= 1.2;  
    } else {  
      console.log("This car does not have a turbocharger.");  
    }  
  }  
}
```

따라서 Car 클래스가 가진 프로퍼티와 메서드를 포함하여,
SportsCar 클래스만의 프로퍼티, 메서드를 더 추가할 수 있다.



상속과 private

```
class SportsCar extends Car {  
  #horsePower;  
  #isTurboCharged;  
  
  boostSpeed() {  
    if (this.#isTurboCharged) {  
      console.log("Boosting speed with turbocharger!");  
      // 예시로 속도를 20% 증가시킴  
      this.#speed *= 1.2; // Error  
    } else {  
      console.log("This car does not have a turbocharger.");  
    }  
  }  
}
```

위 코드의 표시된 부분은 에러가 발생한다.

#speed 프로퍼티는 **private**이기 때문이다.

(자식 클래스도 엄연히 다른 클래스이다.)



상속과 private

```
class Car {
  #name;
  #speed;
  #wheels;

  constructor(name, speed) {
    this.#name = name;
    this.#speed = speed;
    this.#wheels = 4;
  }

  get speed() {
    return this.#speed;
  }

  set speed(speed) {
    if (speed < 0) throw new Error("speed cannot be minus");
    if (typeof speed !== "number") throw new Error("speed must be number");
    this.#speed = speed;
  }
}
```

```
class SportsCar extends Car {
  #horsePower;
  #isTurboCharged;

  boostSpeed() {
    if (this.#isTurboCharged) {
      console.log("Boosting speed with turbocharger!");
      // 예시로 속도를 20% 증가시킴
      this.speed *= 1.2;
    } else {
      console.log("This car does not have a turbocharger.");
    }
  }
}
```

따라서 자식 클래스에서 부모 클래스의 private 프로퍼티에 접근하려면
부모 클래스에서 setter를 설정해줘야 한다.



자식 클래스의 생성자

```
class Car {  
  #name;  
  #speed;  
  #wheels;  
  
  constructor(name, speed) {  
    this.#name = name;  
    this.#speed = speed;  
    this.#wheels = 4;  
  }  
  
  printSpec() {  
    console.log(`${this.#name}/${this.#speed}/${this.#wheels}`);  
  }  
}  
  
class SportsCar extends Car { }  
  
const porsche = new SportsCar("Porsche", 400);  
porsche.printSpec(); // 'Porsche/400/4'
```

자식 클래스는 기본적으로 부모 클래스의 생성자를 사용한다.



자식 클래스의 생성자

```
class SportsCar extends Car {  
  #horsePower;  
  #isTurboCharged;  
  
  constructor(name, speed, horsepower, isTurboCharged) {  
    super(name, speed);  
    this.#horsePower = horsepower;  
    this.#isTurboCharged = isTurboCharged;  
  }  
}  
  
const porsche = new SportsCar("Porsche", 400, 500, true);
```

그러나 자식 클래스만의 추가적인 초기화 작업이 필요하다면,

생성자를 새로 정의할 수 있다.

이때 `super()`를 사용한다.

```
class SportsCar extends Car {  
  #horsePower;  
  #isTurboCharged;  
  
  constructor(name, speed, horsePower, isTurboCharged) {  
    super(name, speed);  
    this.#horsePower = horsePower;  
    this.#isTurboCharged = isTurboCharged;  
  }  
}  
  
const porsche = new SportsCar("Porsche", 400, 500, true);
```

`super()`는 부모 클래스의 생성자를 호출하는 메서드이다.

무조건 자식 클래스 생성자의 맨 첫 줄에서 **`super()`**를 호출해야 한다.


```
class Car {
  #name;
  #speed;
  #wheels;

  constructor(name, speed) {
    this.#name = name;
    this.#speed = speed;
    this.#wheels = 4;
  }

  printSpec() {
    console.log(`${this.#name}/${this.#speed}/${this.#wheels}`);
  }
}
```

```
class SportsCar extends Car {
  #horsePower;
  #isTurboCharged;

  constructor(name, speed, horsePower, isTurboCharged) {
    super(name, speed);
    this.#horsePower = horsePower;
    this.#isTurboCharged = isTurboCharged;
  }

  printSpec() {
    super.printSpec();
    console.log(`horse power: ${this.#horsePower}`);
    console.log(`isTurboCharged: ${this.#isTurboCharged}`);
  }
}

const porsche = new SportsCar("Porsche", 400, 500, true);
porsche.printSpec();
```

‘**super**’ 키워드를 호출하지 않고 **참조**하면,
부모 클래스의 메서드를 호출할 수 있다.


```
class SportsCar extends Car {  
  #horsePower;  
  #isTurboCharged;  
  
  constructor(name, speed, horsepower, isTurboCharged) {  
    super(name, speed);  
    this.#horsePower = horsepower;  
    this.#isTurboCharged = isTurboCharged;  
  }  
  
  printSpec() {  
    super.printSpec();  
    console.log(`horse power: ${this.#horsePower}`);  
    console.log(`isTurboCharged: ${this.#isTurboCharged}`)  
  }  
}  
  
const porsche = new SportsCar("Porsche", 400, 500, true);  
porsche.printSpec();
```

Porsche/400/4
horse power: 500
isTurboCharged: true

‘**super**’ 키워드를 호출하지 않고 **참조**하면,
부모 클래스의 메서드를 호출할 수 있다.



오버라이딩 (Overriding)

```
class Car {  
  #name;  
  #speed;  
  #wheels;  
  
  constructor(name, speed) {  
    this.#name = name;  
    this.#speed = speed;  
    this.#wheels = 4;  
  }  
  
  printSpec() {  
    console.log(`${this.#name}/${this.#speed}/${this.#wheels}`);  
  }  
}
```

```
class SportsCar extends Car {  
  #horsePower;  
  #isTurboCharged;  
  
  constructor(name, speed, horsePower, isTurboCharged) {  
    super(name, speed);  
    this.#horsePower = horsePower;  
    this.#isTurboCharged = isTurboCharged;  
  }  
  
  printSpec() {  
    super.printSpec();  
    console.log(`horse power: ${this.#horsePower}`);  
    console.log(`isTurboCharged: ${this.#isTurboCharged}`)  
  }  
}  
  
const porsche = new SportsCar("Porsche", 400, 500, true);  
porsche.printSpec();
```

부모 클래스에서 정의된 메서드를

자식 클래스에서 **재정의**하는 것을 **오버라이딩(Overriding)**이라고 한다.



오버라이딩 (Overriding)

```
class Car {
  #name;
  #speed;
  #wheels;

  constructor(name, speed) {
    this.#name = name;
    this.#speed = speed;
    this.#wheels = 4;
  }

  printSpec() {
    console.log(`${this.#name}/${this.#speed}/${this.#wheels}`);
  }
}
```

```
class SportsCar extends Car {
  #horsePower;
  #isTurboCharged;

  constructor(name, speed, horsePower, isTurboCharged) {
    super(name, speed);
    this.#horsePower = horsePower;
    this.#isTurboCharged = isTurboCharged;
  }

  printSpec() {
    super.printSpec();
    console.log(`horse power: ${this.#horsePower}`);
    console.log(`isTurboCharged: ${this.#isTurboCharged}`);
  }
}

const porsche = new SportsCar("Porsche", 400, 500, true);
porsche.printSpec();
```

완전히 다른 동작으로 재정의할 수도 있지만,

위와 같이 **부모 클래스의 동작에서 확장**하고자 할 때는

`super`를 참조하여 해당 메서드를 호출하고, 추가 동작을 정의하는 경우가 많다.